# fn match_per_group_predicate

Michael Shoemate (@Shoeboxam)

November 12, 2026

> This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `match_per_group_predicate` in `mod.rs` at commit f5bb719 (outdated[1]).

## 1 Hoare Triple

### Precondition

#### Compiler-verified

Types matching pseudocode.

#### Caller-verified

None

### Pseudocode

```
def match_per_group_predicate(
    enumeration: Expr,
    partition_by: Vec[Expr],
    identifier: Expr,
    threshold: u32,
) -> Optional[Bound]:
    range = int_range(lit(0), len(), 1, DataType.Int64)
    if enumeration == range:
        raise "physical row order is not invariant under unordered identifier metrics"

    trusted = match_permute(enumeration) == range
    one_based = False
    if not trusted:
        trusted = match_ordinal_rank(
            enumeration, range
        )  #
        one_based = trusted
    if not trusted:
        return None

    ids, by = partition(
        lambda expr: expr == identifier, partition_by
    )  #
    if not ids:
```

[1] See new changes with `git diff f5bb719..e8cbf7e11 rust/src/transformations/make_stable_lazyframe/truncate/matching/mod.rs`

```
25          raise "failed to find identifier column in per_group predicate condition"
26      return Bound(
27          by=by, per_group=max(0, threshold - int(one_based)), num_groups=None
28      )  #
29
30
31  def match_ordinal_rank(enumeration: Expr, range: Expr) -> bool:
32      if not (
33          isinstance(enumeration, Expr.Function)
34          and isinstance(enumeration.function, FunctionExpr.Rank)
35          and enumeration.function.options.method == RankMethod.Ordinal
36          and len(enumeration.input) == 1
37          and isinstance(enumeration.input[0], Expr.Function)
38          and enumeration.input[0].function == FunctionExpr.AsStruct
39      ):
40          return False
41
42      fields = enumeration.input[0].input
43      if not fields:
44          return False
45      *keys, last = fields
46      if not isinstance(last, Expr.Alias) or match_permute(last.expr) != range:
47          return False
48
49      for key in keys:  #
50          if not is_rowwise(key):
51              raise "SortBy truncation keys must be rowwise"
52          check_infallible(key, Resize.Ban)
53      return True
```

## Postcondition

**Theorem 1.1** (Postcondition)**.** If `enumeration` is an enumeration of rows, and `partition_by` includes `identifier`, then returns a `threshold` bound on per-group contribution, when grouped by the non-identifier columns in `partition_by`.

*Proof.* The matcher accepts only a full trusted uniform permutation of `int_range(0, len)`, or ordinal ranks of a struct consisting of rowwise sort keys and a final trusted uniform permutation. Line 16 rejects other expressions.

In the sorted case, each row has a distinct random tie-breaking priority. The sort keys follow rows under input permutations and are infallible, as checked on line 49. Thus the resulting selected rows have the same distribution regardless of physical input order.

By line 23, the window partitions rows by identifier and the additional grouping expressions. Full permutations assign distinct values $0, \ldots, n-1$; ordinal ranks assign $1, \ldots, n$. Line 28 accounts for this offset when converting the predicate threshold into a per-group contribution bound. □